Лабораторна робота 3

Вказівники на функції та заголовні файли

1 Завдання на лабораторну роботу

1.1 Виведення таблиці значень функції

Створити окрему одиницю трансляції, в якій реалізувати функцію виведення таблиці значень певної функції з визначеним кроком. Параметри функції – початок, кінець інтервалу, крок і вказівник на функцію, значення якої будуть виведені в таблиці. У заголовному файлі описати тип вказівника на функцію і прототип функції виведення таблиці значень. У файлі реалізації визначити функцію виведення таблиці значень.

В іншій одиниці трансляції розташувати функції, для яких слід вивести значення у таблиці, а також функцію `main()`, в якій здійснюється виведення таблиць значень не менш ніж двох різних функцій. Одна з функцій для тестування – функція, визначена в завданні першої лабораторної роботи та реалізована в другій лабораторній роботі.

2 Методичні вказівки

2.1 Псевдоніми типів

C++ дозволяє створити псевдонім для імені типу, який існує, використовуючи ключове слово **`typedef`**. В результаті створюється синонім для типу. Важливо відрізняти створення синоніма від створення нового типу (визначення структур, перелічень і класів). Визначення синоніму починається з ключового слова **`typedef`**, за яким розташовують існуючий тип, а потім – нове ім'я (ідентифікатор). Наприклад,

```
typedef unsigned long int Cardinal;

typedef int IntArray[15];
```

створює нове ім'я `Cardinal`, яке можна використовувати в будь-якому місці замість **`unsigned long int`**. Ідентифікатор `IntArray` може бути використаний для визначення масиву з 15 цілих значень:

```
Cardinal c;

int f(Cardinal k);

IntArray a; // int a[15];
```

Визначення **`typedef`** інтерпретується таким же чином, як визначення змінної, але ідентифікатор стає синонімом типу.

Визначення `typedef` дозволяє побудувати коротші або більш значущі імена для типів, які вже визначені в мові або для користувацьких типів. Ці імена дозволяють приховати деталі реалізації, які можуть змінитися.

2.2 Вказівники на функції

Вказівник на функцію – це адреса, де зберігається скомпільований код цієї функції, тобто адреса, за якою передається управління, коли ця функція викликається. Так само як ім'я масиву є константним вказівником на перший елемент масиву, ім'я функції можна розглядати як константний вказівник на функцію. Можна оголосити змінну – вказівник, який вказує на функцію, і викликати функцію за допомогою цього вказівника.

Вказівник на функцію повинен вказувати на функцію з відповідним типом результату і сигнатурою. У визначенні

```
int (*funcPtr)(double);
```

`funcPtr` оголошується вказівник, який вказує на функцію, що приймає параметр з рухомою комою і повертає ціле значення. Дужки навколо `*funcPtr` необхідні. Без першої пари дужок це був би прототип функції, яка приймає `double` і повертає вказівник на `int`. Оголошення вказівника на функцію завжди буде містити тип результату й круглі дужки, які вказують типи параметрів.

Вказівнику на функцію можна присвоїти адресу певної функції шляхом присвоєння імені функції без дужок. Тепер вказівник можна використовувати так само як ім'я функції. Вказівник на функцію повинен бути узгоджений з функцією за типом результату і сигнатурою. Наприклад:

```
int round(double x)
{
    return x + 0.5;
}

void main()
{
    int (* funcPtr)(double);
    double y;
    cin >> y;
    funcPtr = round;
    cout << funcPtr(y);
}
```

Вказівник на функцію не потрібно розіменовувати, хоча це можна зробити. Тому, якщо `pFunc` є вказівником на функцію, в який записано адресу реальної функції, можна викликати цю функцію безпосередньо

```
pFunc(x);
```

або

```
(*pFunc)(x);
```

Обидві форми ідентичні.

Для створення більш відповідних імен типів вказівників на функції часто використовують визначення `typedef`:

```
typedef int (*FuncType)(int);

FuncType pf;
```

Можна створити масив указівників на функції.

Вказівники на функції часто використовують як типи параметрів функцій. *Механізм зворотного виклику* (callback) передбачає визначення певної функції, виклик якої здійснюється не безпосередньо в тій частині коду, де вона визначена, а з іншої частини коду, куди можна надіслати вказівник на цю функцію, наприклад, як параметр іншої функції.

Наприклад, є певний універсальний алгоритм, який для своєї роботи вимагає здійснення викликів іншої функції. Функція в цьому випадку виступає як деяка інформація поруч з числовими та іншими аргументами. Це можуть бути різні задачі, наприклад:

- реалізація універсального алгоритму розв'язання рівняння
- знаходження максимумів і мінімумів
- обчислення похідної
- обчислення визначеного інтегралу
- знаходження точок перегину
- обробка події, пов'язаної з діями користувача тощо.

Вказівник на функції в першу чергу використовують саме для реалізації цього механізму.

Наприклад, деяка функція (алгоритм) вимагає іншу функцію як параметр:

```
void someAlgorithm(void (*f)(double))
{
    double z;
    //...
    f(z);
    //...
}
```

В іншій частині коду створюємо необхідну функцію та передаємо її адресу як параметр:

```
void g(double x)
{
    //...
}
```

```
void main()
{
    //...
    someAlgorithm(g);
}
```

Приклад 3.1 ілюструє використання механізму callback для розв'язання рівняння методом ділення відрізку навпіл.

2.3 Використання заголовних файлів

Кожна нетривіальна програма може бути розділена на відносно універсальні частини, які можуть бути використані в кількох проєктах, та частини, специфічні для конкретного проєкту. Універсальні та специфічні частини доцільно зберігати в окремих файлах.

Найпростіший шлях для реалізації цієї ідеї полягає у застосуванні директиви препроцесору `#include`, яка дозволяє перед компіляцією включити вихідний текст з одного файлу всередину іншого.

Препроцесор не здійснює фізичного копіювання змісту файлу. Замість цього препроцесор створює новий вихідний текст у оперативній пам'яті. Цей текст має назву *одиниці трансляції*. Можна також видалити певні частини вихідного тексту за допомогою директив `#define`, `#ifdef` та `#ifndef`. Директива `#define` дозволяє визначити нову змінну препроцесору в будь-якому місці. В іншому місці можна перевірити цей факт за допомогою директив `#ifdef` або `#ifndef`. Наприклад:

```
#define New_Name
...
#ifdef New_Name
// Включаємо цей код в одиницю трансляції
#else
// Не включаємо цього коду в одиницю трансляції
#endif
```

Одиниця трансляції – це результат обробки вихідного тексту препроцесором. Один проєкт може містити кілька одиниць трансляції. Імена, визначені в одних одиницях, повинні бути оголошені в інших одиницях. Неправильне оголошення імені може привести до помилок. Правильним рішенням є створення окремого *заголовного файлу* з необхідними описами. Включення заголовних файлів гарантує точне відтворення оголошень у всіх одиницях трансляції.

У заголовний файл можна включати такі елементи:

- іменовані простори імен
- визначення типів
- оголошення функцій
- визначення функцій з модифікатором `inline`
- оголошення даних (з ключовим словом `extern`),
- визначення констант
- директиви препроцесору
- коментарі.

Заголовні файли не повинні містити

- визначень звичайних функцій
- визначень даних
- безіменних просторів імен.

Традиційно заголовні файли мають розширення `.h`, тоді як файли з реалізацією функцій та визначенням даних мають розширення `.cpp`.

Існують чисельні стандартні заголовні файли з описами класів та функцій. Для включення таких файлів замість лапок слід вживати кутові дужки `<>` для того, щоб препроцесор шукав такі файли в стандартних теках. В іншому випадку препроцесор починає пошук з поточної теки.

2.4 Стражі включення (Include Guards)

Дуже часто у вихідний текст необхідно включати більше одного заголовного файлу. Крім того, дуже часто є необхідність включення тексту одного заголовного файлу в інший заголовний файл. Наприклад, є необхідність включення файлу `f1.h` у файл `f2.h`, файл `f3.h` потребує включення файлів `f1.h` та `f2.h`, а всі ці файли необхідні для компіляції основної програми:

```
//f1.h
...

//f2.h
#include "f1.h"
...

//f3.h
#include "f1.h"
#include "f2.h"
...

//main.cpp
#include "f1.h"
#include "f2.h"
#include "f3.h"
...
```

Препроцесор включає вміст файлу `f1.h` у одиницю трансляції, коли він обробляє файл `main.cpp`. Потім він включає вміст файлу `f2.h`, який також містить включення `f1.h`. Після включення `f3.h` одиниця трансляції містить чотири копії тексту `f1.h` та дві копії `f2.h`. Таке включення не має сенсу, а також може викликати помилки під час компіляції.

Найчастіше цю проблему вирішують за допомогою так званих *стражів включення* – спеціальної конструкції з директив препроцесору. Наприклад, текст заголовного файлу `f1.h` можна організувати в такий спосіб:

```
#ifndef F1_H

#define F1_H

... // увесь вміст файлу

#endif
```

Вперше, коли у програмі згадується включення заголовного файлу, препроцесор здійснює читання першого рядку (`#ifndef`) та включає весь текст у одиницю трансляції, оскільки змінна `F1_H` ще не була визначена. Окрім того, препроцесор визначає цю змінну.

Вдруге та будь-коли потім препроцесор не включає тексту заголовного файлу, оскільки змінна `F1_H` вже визначена.

Ім'я, яке визначається у директиві `#define`, може бути довільним. Важливо тільки, щоб воно було визначене тільки в одному заголовному файлі. В іменах змінних препроцесору не можна застосовувати крапку.

Для створення заголовних файлів у середовищі Visual Studio необхідно виконати такі дії:

- вибирати `Add New Item` у підменю `Project`
- вибирати `Header File (.h)` у вікні `Templates`
- ввести нове ім'я файлу без розширення у полі `Name`
- натиснути кнопку `Add`

Можна додати новий файл реалізації аналогічним чином.

Іноді виникає ідея створити в одній одиниці трансляції глобальну змінну та використовувати її для обміну даними між функціями різних одиниць трансляції. Ідея, яка перша приходить в голову – визначити таку змінну в заголовному файлі:

```
int someValue; // Змінна призначена для обміну даними
```

Таке визначення призводить до помилки. Препроцесор включає заголовний файл в кожну одиницю трансляції і компілятор створює декілька таких змінних. Під час збирання програми виникає конфлікт імен, або створюється своя змінна для кожної одиниці трансляції.

Насправді змінну слід оголосити, а не визначити в заголовному файлі. Оголошення змінної повідомляє, що змінна існує і визначається десь в іншому місці. Оголошення не є визначенням, не приводить до виділення пам'яті. Для оголошення змінної без визначення використовується ключове слово `extern`:

```
extern int someValue; // Змінна буде визначена пізніше
```

Змінну слід визначити в одному (і лише в одному) файлі реалізації. Тепер всі одиниці трансляції мають доступ до однієї змінної.

2.5 Простори імен

Простори імен визначають логічну структуру програми.

Простір імен являє собою іменовану область, яка може містити оголошення і визначення констант, змінних, функцій і типів, а також інших просторів імен. Простори імен дозволяють уникнути конфліктів імен. Простори імен дають механізм для логічного групування оголошень і визначень.

```
namespace MySpace
{
    int  k = 10;
    void f(int n)
    {
        k = n;
    }
}
```

Елементи просторів імен можуть бути визначені окремо від своїх оголошень. Наприклад,

```
namespace MySpace
{
    int  k = 10;
    void f(int n);
}

void MySpace::f(int n)
{
    k = n;
}
```

Простори імен можуть бути вкладені в інші простори імен:

```
namespace FirstSpace
{
    namespace SecondSpace
    {
        ...
    }
}
```

Можна використовувати альтернативне ім'я для ідентифікації простору імен:

```
namespace YourSpace = MySpace;
```

Простори імен можна переривати й відкривати знову для подальшого розширення:

```
namespace FirstSpace
{
    // перша частина
}

... // інші описи

namespace SecondSpace
{
    // інший простір імен
}

namespace FirstSpace
{
    // друга частина
}
```

Оголошення одного простору імен можуть знаходитися в різних файлах.

Безіменні простори імен використовують в програмах, які збирають з декількох одиниць трансляції

Є три способи доступу до елементів простору імен:

- за допомогою явної кваліфікації доступу
- за допомогою `using`-оголошення
- за допомогою `using`-директиви.

Перший варіант дозволяє дістатися кожного імені через ідентифікатор простору і необхідне ім'я всередині простору, розташоване після операції дозволу видимості (`::`). Наприклад:

```
int x = MySpace::k;
```

Другий варіант дозволяє отримати доступ до членів простору імен в індивідуальному порядку з використанням синтаксису `using`-оголошення. Оголошений ідентифікатор додається до локального простору імен:

```
using MySpace::k;
using MySpace::f;
int y = k + f(k);
```

Третій спосіб використовують, якщо є потреба у використанні кількох (або всіх) членів простору імен. За допомогою `using`-директиви ми визначаємо, що всі ідентифікатори в просторі імен знаходяться в глобальній області видимості, починаючи з точки, де визначена `using`-директива. Наприклад:

```
using namespace MySpace;
```

Слід уникати використання цієї директиви через можливі конфлікти імен.

Директива `using` може бути використана, якщо ми хочемо поєднати кілька просторів імен:

```
namespace NewSpace
{
  using namespace FirstSpace;
  using namespace SecondSpace;
}
```

Можна вибрати кілька імен з одного або декількох просторів імен у новому просторі імен за допомогою `using`-оголошення:

```
namespace NewSpace
{
  using OtherSpace::name1;
  using OtherSpace::name2;
}
```

Такий простір імен може бути використаний у різних проєктах.

Простори імен у C++ не приховують дані. Після того, як простір імен було підключено за допомогою директиви `using`, усі імена, оголошені в просторі імен, можуть бути використані як глобальні імена без жодних обмежень.

Більшість компонентів стандартної бібліотеки C++ згрупована в просторі імен `std`. Простір імен `std` містить додаткові простори імен, такі як, наприклад, `std::rel_ops`.

Існує спеціальний різновид просторів імен – так звані *безіменні простори імен* (anonymous namespaces). Наприклад,

```
namespace
{
    int k;
    void f()
    {

    }
}
```

Підключення такого простору неможливе ні в який спосіб. В межах фізичного файлу доступ до імен безіменного простору не обмежується і не вимагає префіксів. Поза межами файлу робота з іменами безіменного простору не можлива.

Створення безіменного простору імен і розташування в ньому функцій і змінних, які необхідні лише в цій одиниці трансляції прискорює роботу лінкера, оскільки він навіть не намагається відшукувати в безіменному просторі імена, вжиті в інших одиницях трансляції. Використання безіменних просторів також зменшує ймовірність конфліктів імен під час компонування.

Безіменні простори імен не можна розташовувати в заголовних файлах.

3 Приклади програм

3.1 Метод ділення відрізку навпіл (дихотомії)

Наведена нижче програма знаходить корінь рівняння за допомогою методу дихотомії. Єдине обмеження використання методу дихотомії полягає у тому, що рівняння повинно мати рівно один корінь на заданому інтервалі.

```
#include <iostream>
#include <cmath>


using std::cout;

using std::endl;

using std::fabs;


typedef double (*FuncType)(double);


// Четвертий аргумент має усталене значення:

double root(FuncType f, double a, double b, double eps = 0.001)

{

    double x;

    do

    {

        x = (a + b) / 2;

        if (f(a) * f(x) > 0)

        {

            a = x;

        }

        else

        {

            b = x;

        }

    }
```

```
    while (b - a > eps);
    return x;
}

double g(double x)
{
    return x * x - 2;
}

void main()
{
    cout << root(g, 0, 6) << endl;
    cout << root(g, 0, 6, 0.00001) << endl;
    cout << root(sin, 1, 4) << endl;
    cout << root(sin, 1, 4, 0.00001) << endl;
}
```

Як видно з наведеного коду, для отримання проміжних точок функції використано механізм зворотного виклику.

3.2 Використання заголовних файлів

Припустимо, ми хочемо створити одиницю трансляції, побудовану з заголовного файлу `SomeFile.h` і файлу реалізації `SomeFile.cpp`.

Спочатку додаємо стражі включення:

```
#ifndef SomeFile_h
#define SomeFile_h


#endif
```

У заголовному файлі перед `#endif` розташовуємо прототип функції:

```
#ifndef SomeFile_h
#define SomeFile_h


int sum(int a, int b);


#endif
```

Файл реалізації містить визначення функції.

```
#include"SomeFile.h"


int sum(int a, int b)
{
    return a + b;
}
```

Тепер заголовний файл `SomeFile.h` слід включити в головну програму:

```
#include <iostream>
#include "SomeFile.h"


using namespace std;


void main()
{
    int x, y;
    cout << "Enter two integer values:" << endl;
    cin >> x >> y;
    int z = sum(x, y);
    cout << "Sum is " << z << endl;
}
```

4 Вправа для контролю

Реалізувати приклади і вправи лабораторної роботи № 3 з розташуванням в окремій одиниці трансляції всіх функцій, крім `main()`.

5 Контрольні запитання

1. Як створити синонім для типу?
2. Що таке вказівник на функцію?
3. Для чого використовують вказівники на функції?
4. Як визначити вказівник на функцію?
5. Що таке одиниця трансляції?
6. Як здійснюється використання директиви #define?
7. Які правила розподілу сирцевого коду між заголовним файлом і файлом реалізації?
8. У чому різниця між включенням стандартних заголовних файлів і заголовних файлів користувача?
9. Що таке стражі включення?

10. Що таке простір імен?
11. Як об'єднати кілька просторів імен в один?
12. Як визначити псевдонім для простору імен?